Art Gallery Management System

Project Report

Automated Software Testing (ATTSW) - A.Y. 2025/2026

Student: Mohamed Jama Farah
Email: mohamed.jama@edu.unifi.it
GitHub: https://github.com/mohamedjama-farah/Art-Gallery-project-
Professor: Lorenzo Bettini
University of Florence - June 2026

1. Introduction

This report describes the Art Gallery Management System, a Java desktop application developed for the Automated Software Testing course. The project was built following Test-Driven Development from the first line of code, with 100% code coverage, zero survived mutants, and zero SonarCloud issues.

The application lets users add, view, and delete artworks through a Swing GUI, with MongoDB as the persistence layer. The goal was not to build a complex domain but to apply every technique taught in the course correctly: TDD, unit/integration/end-to-end testing, mutation testing, continuous integration, and code quality analysis.

The GitHub repository is:

https://github.com/mohamedjama-farah/Art-Gallery-project-

To build and run the project locally:

- Start MongoDB: docker start mymongo (or docker run -d -p 27017:27017 --name mymongo mongo:6.0)
- Run all tests: mvn clean verify
- Run the GUI: mvn exec:java -Dexec.mainClass=com.artgallery.ArtGalleryApp -DskipTests

Java 17, Maven, and Docker must be installed. The tests use Testcontainers so Docker must be running during mvn clean verify.

2. Domain Model

The domain is intentionally simple. There are two entities: Artwork and Category.

2.1 Artwork

An Artwork has four fields: a UUID (generated automatically in the constructor), a title (String), an artist (String), and a price (double). The UUID is generated with UUID.randomUUID() so no two artworks can share the same identifier even if title and artist match. There are no nullable constraints beyond what Java enforces by default.

2.2 Category

A Category has two fields: an id (String) and a name (String). In the current version categories are stored and retrieved independently; the relation between artworks and categories is not implemented in the persistence layer but the domain model and repository interfaces are in place.

Keeping the domain small was deliberate. The course guidelines say complexity of the main code is not what matters. What matters is how you test it. Two entities with a handful of fields give enough surface area to demonstrate TDD, mocking, integration testing with a real database, and end-to-end GUI testing without drowning in domain logic.

3. Architecture and Design

The application follows the MVP (Model-View-Presenter) pattern, which is the standard for Swing applications in this course.

3.1 Packages

- com.artgallery.model - Artwork and Category entity classes
- com.artgallery.view - The ArtGalleryView interface
- com.artgallery.view.swing - ArtGalleryFrame (the Swing implementation)
- com.artgallery.controller - ArtworkController and CategoryController
- com.artgallery.repository - Repository interfaces and MongoDB implementations

3.2 ArtworkController

The controller takes two constructor arguments: an ArtworkRepository and an ArtGalleryView. This makes it fully testable with Mockito - the tests pass mock implementations of both interfaces and verify behavior without touching the database or the UI.

The controller has five methods: allArtworks(), addArtwork(Artwork), deleteArtwork(Artwork), getArtworkById(String), and updateArtwork(Artwork). Each method delegates to the repository and calls the appropriate view method to update the display.

3.3 ArtGalleryFrame

ArtGalleryFrame implements ArtGalleryView and extends JFrame. It uses a no-argument constructor followed by setController(ArtworkController) - this is the pattern shown in the book and it keeps the frame testable without needing a real controller.

The frame has a title text field, an artist text field, a price text field, an Add button, a Delete button, a JList backed by a DefaultListModel, and an error label. The Add button is disabled until all three fields contain non-blank text - this is implemented with a DocumentListener attached to each field.

3.4 Repository Layer

ArtworkRepository and CategoryRepository are interfaces. MongoArtworkRepository and MongoCategoryRepository implement them using the MongoDB Java driver

directly (no Spring Data, no ORM). The MongoClient connection string defaults to localhost:27017.

The repository classes are excluded from pitest mutation testing. This is correct: the repositories are only testable through integration tests that use a real (Testcontainers) MongoDB instance, and pitest runs only unit tests. Mutating repository code that pitest cannot cover would produce false failures.

4. Test Strategy

The project has 197 tests split across three categories. This matches the testing pyramid: many unit tests at the base, fewer integration tests in the middle, and a small number of end-to-end tests at the top.

4.1 Unit Tests (src/test/java) - 144 tests

Unit tests cover the controller classes and the Swing frame. The controllers are tested with Mockito mocks for both the repository and the view. The frame tests use AssertJ-Swing to interact with the GUI in a headless environment.

The frame tests verify things like: the Add button is disabled when any field is empty or whitespace-only; clicking Add calls controller.addArtwork(); clicking Delete with an item selected calls controller.deleteArtwork(); artworkAdded() appends to the list and clears the error label; showError() sets the error label text. This gives complete behavioral coverage of the UI layer without touching MongoDB.

4.2 Integration Tests (src/it/java) - 46 tests

The MongoArtworkRepositoryIT and MongoCategoryRepositoryIT classes test the repository implementations against a real MongoDB instance managed by Testcontainers. Each test class starts a fresh container, runs the tests, and stops the container automatically.

These tests verify that findAll(), findById(), save(), and delete() work correctly with a real database. They also test edge cases like deleting a non-existent document and finding by an ID that does not exist.

4.3 End-to-End Tests (src/it/java) - 7 tests

ArtGalleryE2EIT starts the full application stack: a Testcontainers MongoDB instance, a real MongoArtworkRepository connected to it, a real ArtworkController, and a real ArtGalleryFrame. The tests use AssertJ-Swing to drive the GUI and verify that actions taken in the UI are correctly persisted in the database and reflected back in the display.

For example: type a title, artist, and price; click Add; verify the item appears in the list; click the item; click Delete; verify the item is removed from both the list and the database.

4.4 Test-Driven Development

Every production class was written test-first. The TDD cycle was: write a failing test, write the minimum code to make it pass, refactor. The Git history shows this progression - tests were committed before the corresponding implementation.

5. Build Automation and Continuous Integration

5.1 Maven Build

The Maven build is organized around two phases. The test phase runs the unit tests via the Surefire plugin. The integration-test and verify phases run the integration and E2E tests via the Failsafe plugin.

The build-helper-maven-plugin adds src/it/java as a test source directory so that the IT tests are compiled and run in the correct phase. Without this, Failsafe would not find the integration test classes.

JaCoCo is configured with two executions: one that generates a coverage report after the unit tests (report-unit), and one that generates the full report after the integration tests (report). This gives SonarCloud the combined coverage from both test runs. ArtGalleryApp is excluded from JaCoCo coverage because it contains only a main() method that launches the GUI - testing it would require mocking the JVM startup.

5.2 GitHub Actions

The CI workflow runs on every push to main. It uses Ubuntu and sets up Java 17 via the Temurin distribution. Docker is available on GitHub Actions runners by default, which is required for Testcontainers.

The build runs mvn clean verify with xvfb-run to provide a virtual display for the AssertJ-Swing tests. Without xvfb the Swing tests would fail because there is no display server on a headless CI runner.

After the build succeeds, the workflow uploads the JaCoCo report to Coveralls and runs the SonarCloud scan. Both are configured with the jacoco.xml report path so they receive accurate coverage data.

5.3 SonarCloud

The SonarCloud project key is mohamedjama-farah_Art-Gallery-project-. The quality gate passes with: Security A, Reliability A, Maintainability A (0 issues), Coverage 100%, and 0% duplications.

Automatic Analysis is disabled on SonarCloud so that the scanner runs from Maven during CI rather than from SonarCloud's own analysis engine. This is required to pass the coverage data correctly.

6. Code Quality

6.1 Mutation Testing

Pitest is configured to target only the controller classes: com.artgallery.controller.*. The repository classes and the Swing frame are excluded from mutation testing.

Excluding the repositories is correct because they are only covered by integration tests, and Pitest runs only unit tests. Mutating code that no unit test covers would generate 'no coverage' mutations, which are not useful for measuring test quality.

With this configuration, Pitest generates 24 mutations targeting the controller logic. All 24 are killed. Line coverage is 100% for the mutated classes. Test strength is 100%.

6.2 SonarCloud Issues

The project reached zero SonarCloud issues after fixing a small number of maintainability warnings. The fixes involved: replacing JFrame.EXIT_ON_CLOSE with WindowConstants.EXIT_ON_CLOSE (which avoids importing a constant from a class that is not otherwise needed), removing duplicate test methods that were identified as cognitive complexity issues, and consolidating parameterized test data providers that had overlapping cases.

6.3 Code Formatting

The code uses consistent indentation throughout. Spaces and tabs are not mixed. The formatting follows the Eclipse default Java formatter, which is what the professor's import of the Eclipse project will see.

7. Problems Encountered and Solutions

7.1 Pitest and Integration Tests

The first time I ran Pitest, it reported 84 generated mutations with 60 having no coverage. The problem was that the targetClasses configuration included the repository package. Repository methods are only called from integration tests, which Pitest does not run. The solution was to remove com.artgallery.repository.* from targetClasses entirely, leaving only the controller package. After this change Pitest generated 24 mutations and killed all of them.

7.2 SonarCloud Coverage

SonarCloud initially showed 0% coverage even though JaCoCo was producing a report locally. The problem was that Automatic Analysis was still enabled on SonarCloud, which means SonarCloud was running its own analysis and ignoring the coverage data from the Maven scan. Disabling Automatic Analysis in the SonarCloud project settings fixed this immediately.

7.3 AssertJ-Swing on GitHub Actions

The Swing tests passed locally but failed on GitHub Actions with a HeadlessException. The fix is xvfb-run in the CI workflow step. This starts a virtual X11 display server that Swing can connect to, even though there is no physical screen. The command is: xvfb-run --auto-servernum mvn clean verify -B.

7.4 JaCoCo Aggregate Report

Getting a single JaCoCo report that covers both unit and integration tests took some iteration. The solution is two JaCoCo executions bound to different Maven phases. The first (report-unit) runs after the test phase and covers unit tests only. The second (report) runs after post-integration-test and covers all tests. The Coveralls and SonarCloud steps both point to the second report, which has the combined coverage.

8. Interesting Development Notes

8.1 Testing the DocumentListener

The Add button disabled/enabled logic is driven by a DocumentListener attached to all three text fields. Testing this is less obvious than testing button clicks. The approach I used was to call setText() directly on the JTextField from the test (not through the GUI action), which fires the DocumentListener as if the user typed. This lets you verify the button state after each change without simulating keystrokes.

One edge case worth noting: the button should be disabled when a field contains only whitespace. The test for this sets the price field to ' ' (three spaces) and checks that the Add button is still disabled. The implementation uses String.trim().isEmpty() for this check.

8.2 Testcontainers and Test Isolation

Each integration test class gets its own MongoDB container. The container starts once per test class (using @BeforeAll) and each test method clears the collection before running. This gives isolation between tests without the overhead of starting a new container for every test. Testcontainers handles pulling the image, starting the container, mapping the port, and stopping it after the class is done.

8.3 The No-Arg Constructor Pattern for Swing

ArtGalleryFrame has a no-argument constructor and a separate setController() method. The alternative would be to pass the controller in the constructor, but that makes the frame harder to test with AssertJ-Swing because you have to provide a real or mock controller just to instantiate the frame. With setController(), the frame tests can create the frame, verify its initial state, and then call setController(mockController) to inject a mock before testing controller interactions. This is the pattern shown in the book.

9. Conclusion

The Art Gallery Management System applies all the techniques from the course: TDD, unit testing with mocks, integration testing with Testcontainers, end-to-end GUI testing with AssertJ-Swing, mutation testing with Pitest, code quality with SonarCloud, coverage with JaCoCo and Coveralls, and continuous integration with GitHub Actions.

The final state of the project:

- 144 unit tests, 46 integration tests, 7 end-to-end tests - all passing
- 100% code coverage on both Coveralls and SonarCloud
- 0 survived mutants (24 generated, 24 killed, 100% test strength)
- 0 SonarCloud issues, 0 duplicated code
- GitHub Actions CI green on every push

The project is available as a public GitHub repository and is set up as an Eclipse project with all metadata files committed. The build can be reproduced with a single command: mvn clean verify (with Docker running).